FATIMA JINNAH WOMEN UNIVERSITY

Group Members

Amna Abid - 007

Arshiya Shakeel – 017

Ayesha Imran - 019

Submitted to

Madam Madiha

Course

Parallel and Distributed System

Project Name

Quantum Computing Simulation using IBM Qiskit

Our project shows:

How quantum computing works (simulated)
How it is different from classical computing
How it can solve problems faster in some cases

1- **Introduction:**

This project simulates quantum computing using Qiskit, demonstrating quantum phenomena such as superposition, entanglement, and quantum algorithms.

2- **Objective:**

The objective of this project is to simulate quantum circuits using IBM Qiskit and demonstrate the basic principles of quantum computing such as superposition, entanglement, and quantum algorithms. The project also compares quantum computing concepts with classical computing to understand their advantages.

3- TOOLS YOU ARE USING

Python
Anaconda
Spyder
Qiskit (IBM quantum library)
Matplotlib (graphs)

4- METHODS:

- Superposition
- Entanglement
- Quantum coin flip
- Deutsch-Jozsa algorithm
- Classical vs Quantum comparison

Superposition:

A quantum bit (qubit) was placed in a state where it represents both 0 and 1 at the same time until measurement.

Entanglement:

Two qubits were entangled so that the state of one qubit directly affects the state of the other, even when separated.

Deutsch-Jozsa Algorithm:

This algorithm was implemented to demonstrate how quantum computing can solve certain problems faster than classical computing by determining whether a function is constant or balanced in a single step.

WORKING:

STEP 1: Install Qiskit

Open Anaconda Prompt:

pip install qiskit qiskit-aer matplotlib

```
(base) C:\Users\ramzan>pip install qiskit qiskit-aer matplotlib notebook
Collecting qiskit
  Downloading qiskit-2.4.1-cp310-abi3-win_amd64.whl.metadata (13 kB)
Collecting qiskit-aer
  Downloading qiskit_aer-0.17.2-cp311-cp311-win_amd64.whl.metadata (8.5 kB)
Requirement already satisfied: matplotlib in c:\users\ramzan\anaconda3\lib\site-packages (3.8.0)
Requirement already satisfied: notebook in c:\users\ramzan\anaconda3\lib\site-packages (7.0.8)
Collecting rustworkx>=0.15.0 (from qiskit)
  Downloading rustworkx-0.17.1-cp39-abi3-win_amd64.whl.metadata (10 kB)
Requirement already satisfied: numpy<3,>=1.21 in c:\users\ramzan\anaconda3\lib\site-packages (from qiskit) (1.26.4)
Requirement already satisfied: scipy>=1.5 in c:\users\ramzan\anaconda3\lib\site-packages (from qiskit) (1.11.4)
Requirement already satisfied: dill>=0.3 in c:\users\ramzan\anaconda3\lib\site-packages (from qiskit) (0.3.7)
Collecting stevedore>=3.0.0 (from qiskit)
  Downloading stevedore-5.7.0-py3-none-any.whl.metadata (2.4 kB)
Requirement already satisfied: typing-extensions in c:\users\ramzan\anaconda3\lib\site-packages (from qiskit) (4.9.0)
Requirement already satisfied: psutil>=5 in c:\users\ramzan\anaconda3\lib\site-packages (from qiskit-aer) (5.9.0)
Requirement already satisfied: python-dateutil>=2.8.0 in c:\users\ramzan\anaconda3\lib\site-packages (from qiskit-aer) (2.8.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\ramzan\anaconda3\lib\site-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in c:\users\ramzan\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\ramzan\anaconda3\lib\site-packages (from matplotlib) (4.25.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\ramzan\anaconda3\lib\site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\ramzan\anaconda3\lib\site-packages (from matplotlib) (23.1)
Requirement already satisfied: pillow>=6.2.0 in c:\users\ramzan\anaconda3\lib\site-packages (from matplotlib) (10.2.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\ramzan\anaconda3\lib\site-packages (from matplotlib) (3.0.9)
Requirement already satisfied: jupyter-server<3,>=2.4.0 in c:\users\ramzan\anaconda3\lib\site-packages (from notebook) (2.10.0)
Requirement already satisfied: jupyterlab-server<3,>=2.22.1 in c:\users\ramzan\anaconda3\lib\site-packages (from notebook) (2.25.1)
Requirement already satisfied: jupyterlab<4.1,>=4.0.2 in c:\users\ramzan\anaconda3\lib\site-packages (from notebook) (4.0.11)
Requirement already satisfied: notebook-shim<0.3,>=0.2 in c:\users\ramzan\anaconda3\lib\site-packages (from notebook) (0.2.3)
Requirement already satisfied: tornado>=6.2.0 in c:\users\ramzan\anaconda3\lib\site-packages (from notebook) (6.3.3)
Requirement already satisfied: anyio>=3.1.0 in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (4.2.0)
Requirement already satisfied: argon2-cffi in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (21.3.0)
Requirement already satisfied: jinja2 in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (3.1.3)
Requirement already satisfied: jupyter-client>=7.4.4 in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (8.
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook
Requirement already satisfied: jupyter-events>=0.6.0 in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (0.
Requirement already satisfied: jupyter-server-terminals in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook)
Requirement already satisfied: nbconvert>=6.4.4 in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (7.10.0)
Requirement already satisfied: nbformat>=5.3.0 in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (5.9.2)
Requirement already satisfied: overrides in c:\users\ramzan\anaconda3\lib\site-packages (from jupyter-server<3,>=2.4.0->notebook) (7.4.0)
```

```
0->jupyter-server<3,>=2.4.0->notebook) (2.1)
Collecting uri-template (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->notebook)
  Downloading uri_template-1.3.0-py3-none-any.whl.metadata (8.8 kB)
Collecting webcolors>=1.11 (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.6.0->jupyter-server<3,>=2.4.0->notebook)
  Downloading webcolors-25.10.0-py3-none-any.whl.metadata (2.2 kB)
Requirement already satisfied: cffi>=1.0.1 in c:\users\ramzan\anaconda3\lib\site-packages (from argon2-cffi-bindings->argon2-c
>notebook) (1.16.0)
Requirement already satisfied: soupsieve>1.2 in c:\users\ramzan\anaconda3\lib\site-packages (from beautifulsoup4->nbconvert>=6
->notebook) (2.5)
Requirement already satisfied: pycparser in c:\users\ramzan\anaconda3\lib\site-packages (from cffi>=1.0.1->argon2-cffi-binding
<3,>=2.4.0->notebook) (2.21)
Requirement already satisfied: parso<0.9.0,>=0.8.0 in c:\users\ramzan\anaconda3\lib\site-packages (from jedi>=0.16->ipython>=7
.1,>=4.0.2->notebook) (0.8.3)
Requirement already satisfied: wcwidth in c:\users\ramzan\anaconda3\lib\site-packages (from prompt-toolkit<3.1.0,>=3.0.41->ipy
erlab<4.1,>=4.0.2->notebook) (0.2.5)
Requirement already satisfied: arrow>=0.15.0 in c:\users\ramzan\anaconda3\lib\site-packages (from isoduration->jsonschema[form
ents>=0.6.0->jupyter-server<3,>=2.4.0->notebook) (1.2.3)
Requirement already satisfied: executing in c:\users\ramzan\anaconda3\lib\site-packages (from stack-data->ipython>=7.23.1->ipy
->notebook) (0.8.3)
Requirement already satisfied: asttokens in c:\users\ramzan\anaconda3\lib\site-packages (from stack-data->ipython>=7.23.1->ipy
->notebook) (2.0.5)
Requirement already satisfied: pure-eval in c:\users\ramzan\anaconda3\lib\site-packages (from stack-data->ipython>=7.23.1->ipy
->notebook) (0.2.2)
Downloading qiskit-2.4.1-cp310-abi3-win_amd64.whl (9.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.1/9.1 MB 126.2 kB/s eta 0:00:00
Downloading qiskit_aer-0.17.2-cp311-cp311-win_amd64.whl (9.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.6/9.6 MB 144.4 kB/s eta 0:00:00
Downloading rustworkx-0.17.1-cp39-abi3-win_amd64.whl (2.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 114.2 kB/s eta 0:00:00
Downloading stevedore-5.7.0-py3-none-any.whl (54 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 54.5/54.5 kB 257.2 kB/s eta 0:00:00
Downloading webcolors-25.10.0-py3-none-any.whl (14 kB)
Downloading fqdn-1.5.1-py3-none-any.whl (9.1 kB)
Downloading isoduration-20.11.0-py3-none-any.whl (11 kB)
Downloading uri_template-1.3.0-py3-none-any.whl (11 kB)
Installing collected packages: webcolors, uri-template, stevedore, rustworkx, fqdn, qiskit, qiskit-aer, isoduration
Successfully installed fqdn-1.5.1 isoduration-20.11.0 qiskit-2.4.1 qiskit-aer-0.17.2 rustworkx-0.17.1 stevedore-5.7.0 uri-temp
```

STEP 2: Open Spyder

Create new file:

Quantum_project.py

CODE:

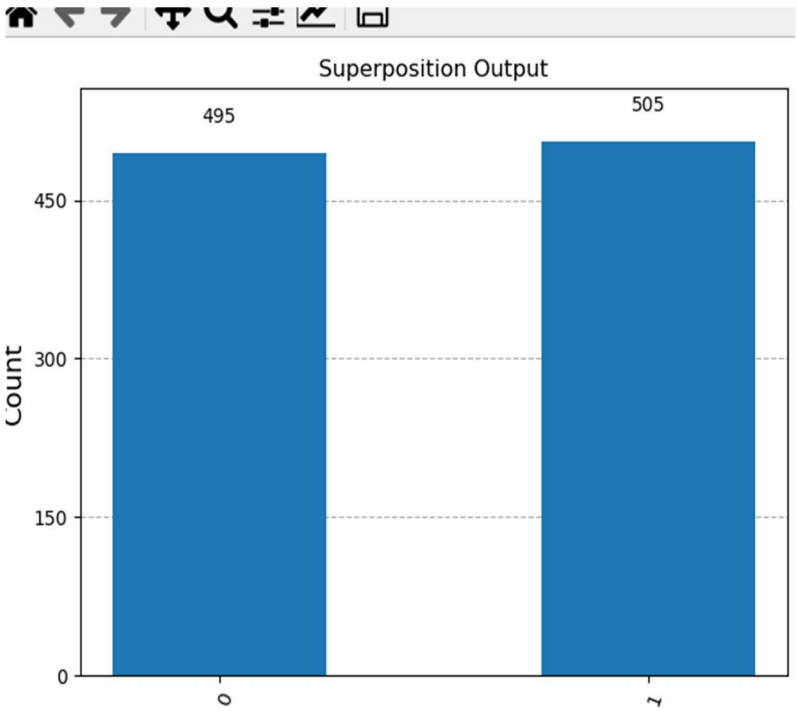
```
print("\nQuantum Computing Simulation using Qiskit\n")

# ===== IMPORTS =====

from qiskit import QuantumCircuit, transpile

from qiskit_aer import Aer

from qiskit.visualization import plot_histogram

import matplotlib.pyplot as plt

import random

# Force interactive plotting (important for Spyder)

plt.ion()

# Simulator backend

sim = Aer.get_backend('qasm_simulator')
```

PART 1: SUPERPOSITION

```
print("\n===== SUPERPOSITION =====")

qc1 = QuantumCircuit(1, 1)

qc1.h(0)

qc1.measure(0, 0)

compiled1 = transpile(qc1, sim)

job1 = sim.run(compiled1, shots=1000)

result1 = job1.result()

counts1 = result1.get_counts()

print("Superposition Result:", counts1)

plot_histogram(counts1)
```

```
plt.title("Superposition Output")

plt.tight_layout()

plt.show(block=True)
```

```
Quantum Computing Simulation using Qiskit


===== SUPERPOSITION =====
Superposition Result: {'0': 490, '1': 510}
```

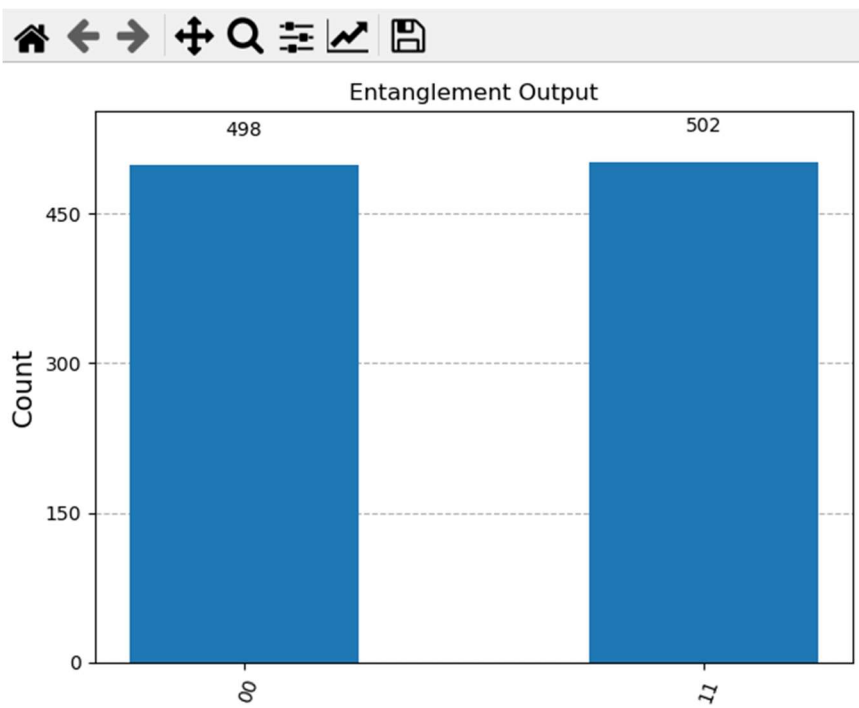


PART 2: ENTANGLEMENT

```
print("\n===== ENTANGLEMENT =====")

qc2 = QuantumCircuit(2, 2)

qc2.h(0)

qc2.cx(0, 1)

qc2.measure([0, 1], [0, 1])
```

```
compiled2 = transpile(qc2, sim)

job2 = sim.run(compiled2, shots=1000)

result2 = job2.result()

counts2 = result2.get_counts()

print("Entanglement Result:", counts2)

plot_histogram(counts2)

plt.title("Entanglement Output")

plt.tight_layout()

plt.show(block=True)
```

```
===== ENTANGLEMENT =====
Entanglement Result: {'00': 521, '11': 479}
```

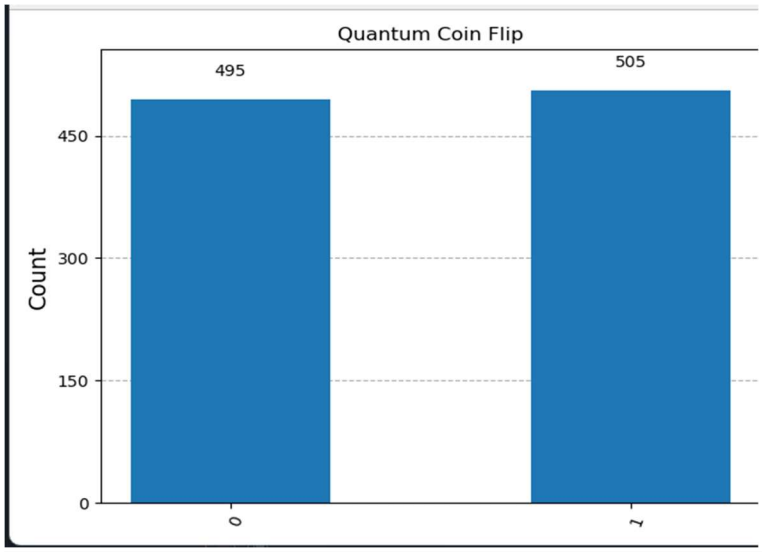


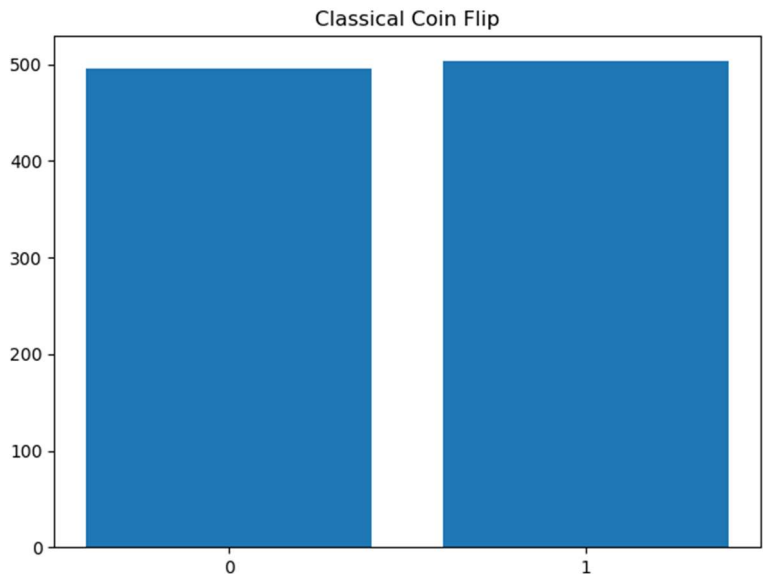
PART 3: QUANTUM vs CLASSICAL COIN FLIP

```
print("\n===== COIN FLIP COMPARISON =====")
```

```
# Classical coin flip
classical_results = [random.choice([0, 1]) for _ in range(1000)]
classical_counts = {
    "0": classical_results.count(0),
    "1": classical_results.count(1)
}
print("Classical Result:", classical_counts)
print("Quantum Result:", counts1)
# Classical plot
plt.figure()
plt.bar(classical_counts.keys(), classical_counts.values())
plt.title("Classical Coin Flip")
plt.tight_layout()
plt.show(block=True)
# Quantum plot
plot_histogram(counts1)
plt.title("Quantum Coin Flip")
plt.tight_layout()
plt.show(block=True)
```

```
===== COIN FLIP COMPARISON =====
Classical Result: {'0': 505, '1': 495}
Quantum Result: {'0': 490, '1': 510}
```



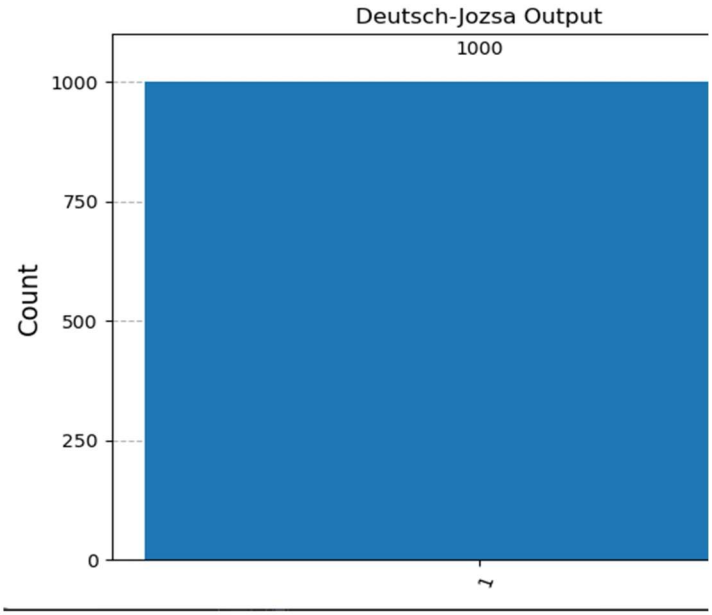


PART 4: DEUTSCH-JOZSA ALGORITHM

```
print("\n===== DEUTSCH-JOZSA ALGORITHM =====")

qc3 = QuantumCircuit(2, 1)

qc3.x(1)

qc3.h([0, 1])

qc3.cx(0, 1)

qc3.h(0)

qc3.measure(0, 0)
```

```
compiled3 = transpile(qc3, sim)

job3 = sim.run(compiled3, shots=1000)

result3 = job3.result()

counts3 = result3.get_counts()

print("Deutsch-Jozsa Result:", counts3)

plot_histogram(counts3)

plt.title("Deutsch-Jozsa Output")

plt.tight_layout()

plt.show(block=True)
```

```
===== DEUTSCH-JOZSA ALGORITHM =====
Deutsch-Jozsa Result: {'1': 1000}
```

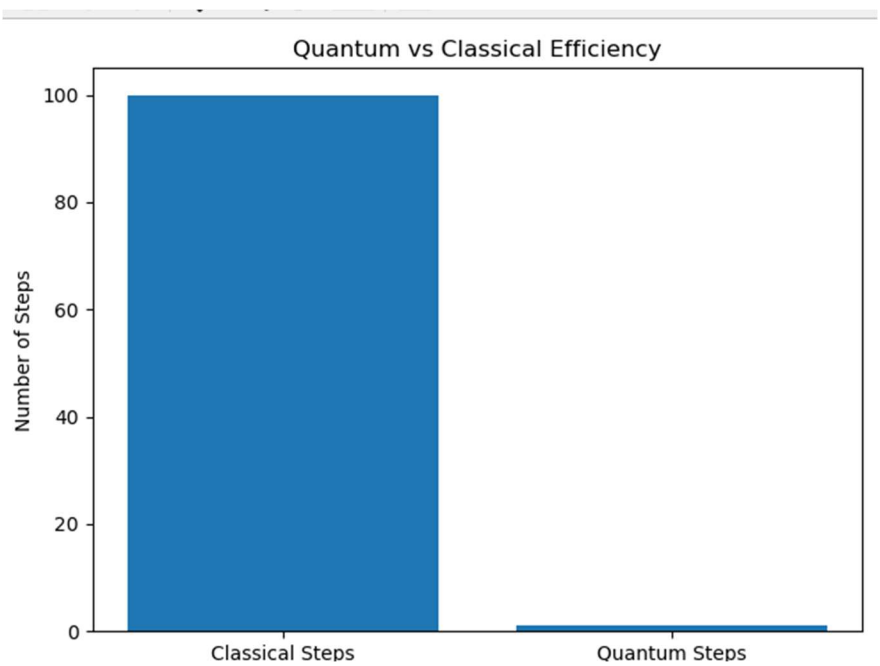


PART 5: PERFORMANCE COMPARISON

```
print("\n===== PERFORMANCE COMPARISON =====")

labels = ['Classical Steps', 'Quantum Steps']
```

```
values = [100, 1]

plt.figure()

plt.bar(labels, values)

plt.title("Quantum vs Classical Efficiency")

plt.ylabel("Number of Steps")

plt.tight_layout()

plt.show(block=True)

# END

print("\n✅ Project Execution Completed Successfully!\n")
```

```
===== PERFORMANCE COMPARISON =====

✅ Project Execution Completed Successfully!

((IPdb [2])):
```



5- Result:

The simulation results showed that:

- Quantum systems produce probabilistic outputs instead of fixed results
- Qubits can exist in multiple states simultaneously (superposition)
- Entangled qubits show strong correlation
- Quantum algorithms can reduce computational steps compared to classical methods

Graphs generated using matplotlib clearly visualized probability distributions of quantum states.

6- The system is parallel or distributed:

This project relates to distributed computing because quantum computing processes multiple possibilities at the same time using superposition.

This is similar to parallel processing where multiple computations occur simultaneously, improving efficiency for specific problems.

7- Conclusion:

Quantum computing is a powerful computing model that uses quantum mechanical principles to perform computations more efficiently for certain problems. This project demonstrates that quantum systems can outperform classical systems in specific computational tasks by leveraging superposition and entanglement.

8- Future Scope:

- Implementation on real IBM Quantum hardware
- Development of advanced quantum algorithms
- Integration with artificial intelligence systems
- Optimization of complex computational problems